mathematics

MDPI

Article

A Comprehensive Approach to Rustc Optimization Vulnerability Detection in Industrial Control Systems

Kaifeng Xie †, Jinjing Wan †, Lifeng Chen † and Yi Wang *

Department of Anthropology and Human Genetics, Fudan University, Shanghai 200433, China; 22210240183@m.fudan.edu.cn (K.X.); wanjinjing@fudan.edu.com (J.W.); chenlf@fudan.edu.cn (L.C.)
* Correspondence: yiwang_@fudan.edu.cn; Tel.: +86-133-6185-7816
† These authors contributed equally to this work.

Abstract

Compiler optimization is a critical component for improving program performance. However, the Rustc optimization process may introduce vulnerabilities due to algorithmic flaws or issues arising from component interactions. Existing testing methods face several challenges, including high randomness in test cases, inadequate targeting of vulnerability-prone regions, and low-quality initial fuzzing seeds. This paper proposes a test case generation method based on large language models (LLMs), which utilizes prompt templates and optimization algorithms to generate a code relevant to specific optimization passes, especially for real-time control logic and safety-critical modules unique to the industrial control field. A vulnerability screening approach based on static analysis and rule matching is designed to locate potential risk points in the optimization regions of both the MIR and LLVM IR layers, as well as in unsafe code sections. Furthermore, the targeted fuzzing strategy is enhanced by designing seed queues and selection algorithms that consider the correlation between optimization areas. The implemented system, RustOptFuzz, has been evaluated on both custom datasets and real-world programs. Compared with state-of-the-art tools, RustOptFuzz improves vulnerability discovery capabilities by 16%–50% and significantly reduces vulnerability reproduction time, thereby enhancing the overall efficiency of detecting optimization-related vulnerabilities in Rustc, providing key technical support for the reliability of industrial control systems.

Keywords: compiler optimization vulnerabilities; test case generation; static analysis; directed fuzz testing; Rustc

MSC: 68T37; 68Q60

Academic Editor: Antanas Cenys

Received: 14 June 2025
Revised: 20 July 2025
Accepted: 21 July 2025
Published: 30 July 2025

Citation: Xie, K.; Wan, J.; Chen, L.; Wang, Y. A Comprehensive Approach to Rustc Optimization Vulnerability Detection in Industrial Control Systems. *Mathematics* **2025**, *13*, 2459. https://doi.org/10.3390/math13152459

1. Introduction

Compiler optimization is a critical step in software development. It aims to improve program performance by transforming a source code written in high-level programming languages into a more efficient machine code [1]. In the field of industrial control, ranging from automated production lines in smart factories to monitoring systems in energy grids, the efficiency of compiler optimizations directly influences the real-time responsiveness and secure operation of these systems. However, incorrect optimizations may result in unexpected program behavior or introduce serious security risks [2]. In recent years, Rust has rapidly gained popularity in areas such as system programming and embedded development because of its memory safety, high performance, and concurrency-friendly

features [3]. It has also been increasingly adopted in core scenarios within industrial control. Although Rust has a robust language design, the optimization techniques applied in its compiler, Rustc, are complex and varied. As a result, detecting vulnerabilities related to these optimizations presents significant challenges. Current testing methods face three major problems in identifying such vulnerabilities. First, test case generation often exhibits randomness and lacks relevance to specific optimization passes. Second, there is no effective mechanism to identify vulnerable optimization regions. Third, the initial seed inputs for fuzz testing are of low quality and fail to reflect correlations with optimization behaviors. These limitations highlight the importance of developing effective techniques for detecting optimization vulnerabilities in Rustc.

To address these issues, this paper presents a new detection method for Rustc optimization vulnerabilities. The method combines large language models, static analysis, and targeted fuzz testing to enhance the efficiency and accuracy of detection. The main contributions of this work are as follows:

1. A compiler optimization pass test case generation method based on a large language model is proposed. By designing a prompt word template and combining the logic information of the optimization pass and the characteristics of the Rust language, a variety of test cases are generated, which significantly improves the relevance of the test cases to the optimization pass.
2. A potential vulnerability screening method based on static analysis and rule matching is proposed. By analyzing the changes before and after the optimization of the Rust intermediate language MIR and LLVM IR, the code area affected by the optimization is accurately located, effectively reducing invalid tests.
3. A directed fuzz testing method based on the correlation of the optimization area is proposed. The optimization-related code snippets are extracted through the context-aware code slicing algorithm, and the high-quality initial seeds are generated in combination with the large language model. The seed queue management mechanism and seed selection strategy are redesigned, which significantly improves the detection efficiency and accuracy of fuzz testing for optimization-related vulnerabilities.

2. Background and Key Insights

2.1. Background

In the context of the in-depth empowerment of the industrial control field in the information society, software has become the core pillar of key industrial control systems such as intelligent manufacturing, energy grids, and rail transit. As a basic tool for industrial software development, the compiler's optimization technology directly determines the real-time response capability and security boundary of the industrial control system. In recent years, the Rust language has gradually penetrated into the core areas of industrial control with its memory safety, zero-cost abstraction, and deterministic execution. In industrial automation scenarios, Rust is used to develop the underlying driver of PLC (programmable logic controller), and its ownership system can effectively avoid common memory safety vulnerabilities in traditional languages; in the field of automotive electronics, Rust's concurrency model provides reliable memory management guarantees for multi-sensor data processing in automotive safety systems. With the expansion of Rust's application in industrial control scenarios, the complexity of its compiler optimization technology has increased exponentially. Although Rustc improves reliability through a strict formal verification mechanism, vulnerabilities may still be introduced during the optimization process due to algorithm defects or component interaction problems, posing a potential threat to the stability of industrial control systems.

The architecture of Rustc can be divided into three parts: the front end, the middle end, and the back- end. The front end is mainly responsible for lexical parsing and syntax parsing tasks, converting a source code into an abstract syntax tree and generating a high-level intermediate representation (HIR); the middle end uses the HIR generated by the front end as input to generate a middle-level intermediate representation (MIR), and performs Rust-specific optimizations; the back end converts the optimized MIR into a low-level virtual machine intermediate representation (LLVM IR), performs optimization operations for the target architecture at the LLVM IR level, and finally generates a machine code suitable for a specific target architecture. Rustc contains dozens or hundreds of analysis and optimization techniques [4]. Although the optimization level (such as 'opt-level = (0–3, s, z)') helps developers choose different optimization techniques to a certain extent, the default optimization level is difficult to fit a variety of programs and complex application scenarios, and cannot achieve optimal performance for all programs. Compiler optimization is a complex and technology-intensive process [5]. Its complexity and the interaction between different optimization techniques make the optimization part the most prone to failure in the compiler. For example, the proportion of optimization-related failures in GCC and LLVM is significant, and there are also many problems with Rustc's MIR and LLVM IR optimizations. These optimization vulnerabilities are caused by developers' coding logic errors or defects in the optimization algorithm itself [6]. They not only affect the usability of the compiler, but also increase the complexity of developers' debugging. In safety-critical areas, they may lead to serious safety accidents. Therefore, it is extremely necessary to develop efficient testing technology to ensure its reliability [7,8].

2.2. Key Insights

In response to the limitations of existing approaches, this paper proposes a novel detection technique for identifying vulnerabilities introduced by Rustc optimizations. By integrating large language models (LLMs), static analysis, and directed fuzz testing, the proposed method significantly enhances both the efficiency and the accuracy of detecting such vulnerabilities. The core methodology is outlined as follows:

1. **Application of large language models:** LLMs have achieved remarkable success in natural language processing and are capable of generating diverse code snippets and understanding code semantics. This paper leverages these capabilities to design a test case generation method based on prompt templates. By embedding logical information about optimization passes and Rust-specific language features into the prompts, the LLM is guided to produce test cases closely aligned with specific optimization passes, thereby improving both the quality and relevance of the generated test cases.
2. **Static analysis and rule matching:** Static analysis techniques can precisely analyze the structural and semantic properties of a code, while rule matching enables rapid identification of potential vulnerability patterns. This paper integrates these two techniques to locate code regions affected by optimizations by analyzing the differences in Rust's intermediate representations MIR and LLVM IR before and after the optimization. In addition, for unsafe code blocks in Rust, rule matching is used to detect regions that may introduce undefined behavior as a result of optimization, thereby reducing invalid tests and enhancing the efficiency of vulnerability detection.
3. **Optimization of targeted fuzz testing:** Targeted fuzz testing enables goal-oriented exploration by focusing on optimization-related code regions, filtering irrelevant inputs, and prioritizing coverage of optimization paths that are more likely to introduce errors. This paper extracts optimization-relevant code fragments using a context-aware code slicing algorithm and generates high-quality initial seeds with the assistance of a large language model. Furthermore, the seed queue management mechanism and selection

strategy are redesigned to comprehensively consider factors such as the distance between the seed and the target, optimization path coverage, and the correlation across multiple optimization regions. These improvements significantly enhance the efficiency and accuracy of fuzz testing for detecting optimization-related vulnerabilities.

3. Design of RustOptFuzz

3.1. Design Concept

The design philosophy of RustOptFuzz is embodied in the following aspects. First, the system emphasizes a correlation-driven approach, requiring a strong association between test cases and optimization passes. This increases the likelihood of triggering specific optimizations and avoids inefficient coverage caused by purely random generation. Second, RustOptFuzz integrates static and dynamic analysis, accurately identifying potential vulnerability regions through static analysis and rule matching, and then verifying them efficiently using dynamic fuzz testing. In addition, the system prioritizes automation and intelligence, leveraging the powerful code generation and semantic understanding capabilities of large language models to automatically produce high-quality test cases and initial seeds, thereby significantly reducing the need for manual intervention. RustOptFuzz also adopts a targeted and efficient strategy, concentrating on optimization-related paths and high-risk regions to enhance both the efficiency and accuracy of vulnerability detection. Finally, the system architecture features strong scalability and generality, supporting various optimization passes and Rust language features, and allowing for easy adaptation and extension to different compiler versions.

3.2. Overall System Architecture

Figure 1 presents the overall system architecture. RustOptFuzz adopts a modular design, consisting of three core components: the test case generation module, the potential vulnerability screening module, and the targeted fuzz testing module. These modules collaborate to form a complete workflow, encompassing test case generation, vulnerability region identification, and vulnerability verification. The test case generation module is responsible for producing high-quality test cases that are closely related to specific optimization passes, laying the foundation for subsequent analysis and testing. The potential vulnerability screening module leverages static analysis and rule matching to accurately identify optimization-related and high-risk code regions, thereby narrowing the testing scope. The targeted fuzz testing module focuses on the identified optimization areas, automatically generating and filtering high-quality seeds, performing directed mutations and executions, and rapidly homing in on potential vulnerabilities.

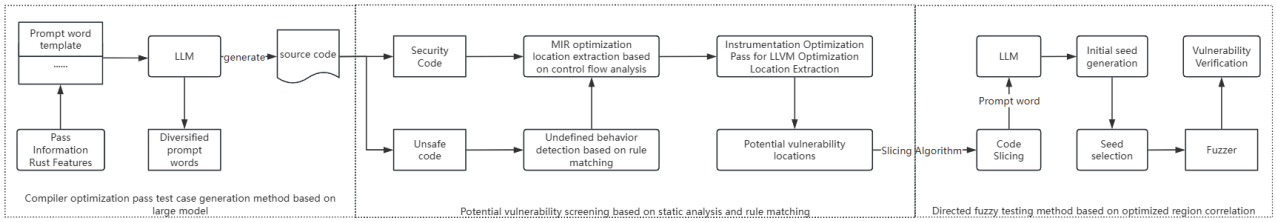

Figure 1. RustOptFuzz framework.

3.3. Design and Implementation of Test Case Generation Module

The test case generation module introduces a method based on an LLM to address issues of excessive randomness and weak correlation with optimization passes found in traditional test case generation approaches. The system designs a structured prompt template that incorporates information such as optimization pass logic, Rust language

features, and task descriptions. This guides the LLM to generate a Rust code that is strongly related to the target optimization pass. In the implementation, the prompt template includes task objectives, functional descriptions of the optimization pass, and required Rust language features. These elements ensure that the generated code meets the conditions necessary to trigger specific optimizations. The system applies low-temperature sampling to generate stable base prompts and high-temperature sampling to increase diversity and improve code coverage. After generating a candidate code using an LLM such as DeepSeek Code, the system automatically compiles and revises the code based on compiler feedback to improve the success rate. It further uses evaluation metrics such as compilation success rate, the number of optimized lines, and a composite score to select high-quality test cases that can compile successfully and trigger the desired optimization. If the code fails to compile, the system analyzes the compiler error messages and automatically adjusts the prompts or code structure accordingly. This process is repeated iteratively to continuously refine and improve the quality and relevance of the generated test cases.

The comprehensive score (Final Score, S) is calculated as Formula (1), and the evaluation function is constructed by linear weighting, as follows:

$$S = \alpha \cdot C + \beta \cdot O_{\text{lines}} \quad \alpha = \beta = 0.5 \tag{1}$$

The screening criteria are as follows:

$$\begin{cases} O_{\text{lines}} \geq 2 & (\geq 2 \text{ optimized code lines}) \\ S \geq 60 & (\text{comprehensive score threshold}) \\ C_{\text{fail}} & (\text{compilation failed}) \end{cases}$$

The large model prompt word optimization algorithm is as follows (Algorithm 1).

Algorithm 1 Prompt word optimization algorithm

Require: Task: `Task`, numOfPrompt: `num`
Ensure: promptList

function GENERATEOPTIMIZEDPROMPTS
  $initialPrompt \leftarrow \text{LLM}(Task, Passes, Features, temp = 0)$
  $candidatePrompts \leftarrow [initialPrompt]$
  while $|candidatePrompts| <$ `num` **do**
    $prompt \leftarrow \text{LLM}(Task, Passes, Features, temp = 1)$
    $candidatePrompts \leftarrow [prompt]$
  end while
  $promptList \leftarrow \{p \in candidatePrompts \mid \text{Scoring}(\text{LLM}(p), SUT) > 60\}$
  return *promptList*
end function

3.4. Design and Implementation of Potential Vulnerability Screening Module

The potential vulnerability screening module proposes a potential vulnerability screening method based on static analysis and rule matching to address the problems of difficulty in locating optimization vulnerabilities and waste of testing resources. The system analyzes the differences before and after IR optimization and marks the code area affected by optimization; it uses rule matching for unsafe code blocks to detect undefined behaviors that may be amplified by optimization. The implementation includes the following components:

- MIR-level screening involves parsing the structure of basic blocks in the Mid-level Intermediate Representation (MIR) before and after optimization, comparing the changes such as additions, deletions, and modifications, and identifying the source code lines affected by the optimization using source mapping information.
- LLVM IR-level screening involves inserting instrumentation into LLVM optimization passes to dynamically record the basic blocks affected by each pass. These blocks are

then mapped back to the source code through debugging information to identify the corresponding optimized lines of a code.

- Detection of an unsafe code via rule matching is based on abstract syntax tree (AST) analysis. The system detects potential undefined behaviors such as pointer operations, integer overflows, and type conversions. These high-risk code segments are then marked through static analysis.

The system integrates results from MIR, LLVM IR, and AST analyses to construct a comprehensive set of potential vulnerability points. These multi-level data provide precise targets for subsequent fuzz testing. The output includes optimized code lines, corresponding optimization passes, and potential undefined behaviors, which helps developers efficiently locate and analyze potential risks.

The MIR potential vulnerability screening algorithm is shown in Algorithm 2.

Algorithm 2 MIR potential vulnerability extraction

Require: MIR files before and after optimization *unopt.mir*and*opt.mir*
Ensure: List of source code lines affected by the optimization
Step 1: Parse the basic block structure
for all MIR File $m \in \{\mathbb{B}_u, \mathbb{B}_o\}$ **do** ▷ - $\mathbb{B}_u$: Set of basic blocks in the unoptimized MIR file (*unopt.mir*) - $\mathbb{B}_o$: Set of basic blocks in the optimized MIR file (*opt.mir*) - m: Iterates over MIR files ($\mathbb{B}_u$ and $\mathbb{B}_o$)
  for all function $f \in m$ **do** ▷ f: Function being processed within the MIR file m
    $\mathbb{B}_f \leftarrow \emptyset$ ▷ $\mathbb{B}_f$: Temporary set to store basic blocks of function f
    for all Basic Blocks $bb \in f$ **do** ▷ bb: A basic block within function f (sequence of instructions with single entry/exit)
      Extract (label, statement, source location, jump target)
      $\mathbb{B}_f \leftarrow \mathbb{B}_f \cup \{bb\}$
    end for
  end for
end for
Step 2: Basic Block Difference Analysis
$\Delta_{\text{add}} \leftarrow \mathbb{B}_o \setminus \mathbb{B}_u$ ▷ Δ_{add}: Set of basic blocks added during optimization ($\mathbb{B}_o$ but not $\mathbb{B}_u$)
$\Delta_{\text{del}} \leftarrow \mathbb{B}_u \setminus \mathbb{B}_o$ ▷ Δ_{del}: Set of basic blocks deleted during optimization ($\mathbb{B}_u$ but not $\mathbb{B}_o$)
$\Delta_{\text{mod}} \leftarrow \{bb \mid H(bb_u) \neq H(bb_o), bb \in \mathbb{B}_u \cap \mathbb{B}_o\}$ ▷ - Δ_{mod}: Set of modified basic blocks - $H(bb)$: Hash function to detect content changes in basic blocks - bb_u / bb_o: Versions of the same basic block in $\mathbb{B}_u / \mathbb{B}_o$
Step 3: Source code line mapping
$L \leftarrow \emptyset$ ▷ L: Set to store source code locations (file path + line number)
for all $bb \in \Delta_{\text{add}} \cup \Delta_{\text{del}} \cup \Delta_{\text{mod}}$ **do**
  for all Statements $s \in bb$ **do** ▷ s: A statement (instruction) within basic block bb
    if s.`has_src_loc()` **then** ▷ s.`has_src_loc()`: Check if statement has associated source code location
      $L \leftarrow L \cup \{(s.\texttt{path}, s.\texttt{line})\}$ ▷ - s.`path`: File path of the source code - s.`line`: Line number in the source code file
    end if
  end for
end for
Step 4: Generate Report
Output $\bigcup_{(p,l)\in L} p : l$,Sort by file group

3.5. Design and Implementation of Directed Fuzz Testing Module

To address the challenge that misoptimization vulnerabilities are often hidden and difficult to trigger, the directed fuzz testing module introduces a multi-level seed priority queue and a context-aware seed generation method. These methods leverage the correlation with optimization areas to improve testing efficiency and increase the vulnerability discovery rate. The specific implementation includes the following components:

- **Initial seed generation:** A context slicing algorithm is used to extract code snippets that contain optimized lines and their associated control flow contexts. Combined with

a large language model, this process generates initial seeds that are highly relevant to the target optimization areas, thereby narrowing the search space.

- **Seed priority queue:** Seeds are categorized into high-, medium-, and low-priority queues based on indicators such as the distance between the seed and the target code line and the coverage rate of optimization paths. Seeds covering more optimization paths and closer to the target are prioritized.
- **Dynamic scheduling and resource allocation:** Using an annealing probability strategy and a dynamic scheduling mechanism, computing resources are allocated primarily to high-priority seeds. This approach enhances the efficiency of exploring critical paths.
- **Rare path and high-quality seed screening:** Seeds capable of triggering rare control flow paths or demonstrating higher execution efficiency are given priority, which helps avoid wasting resources on redundant paths.

The context-aware slicing algorithm is shown in Algorithm 3.

Algorithm 3 Context-aware slicing algorithm

Require: Code Blocks C, Optimizing Rowsets $O = \{l_1, \ldots, l_m\}$
Ensure: Slice Collection $S \subseteq C$

$S \leftarrow \emptyset$
for $\forall l_k \in O$ **do**
  $B_i \leftarrow \phi(l_k), \phi : L \rightarrow B$ ▷ Row to basic block mapping
  analyze $\Psi(B_i), \Psi(B) \in \{0, 1\}$ ▷ Boundary detection function
  if $\Psi(B_i) = 1$ **then**
    $\partial B_i \leftarrow \Gamma(B_i), \Gamma(B) = \cup N(B)$ ▷ Closure Operation
    $S \leftarrow S \cup \partial B_i$
  else
    $S \leftarrow S \cup \{B_i\}$
  end if
end for
return S

The seed generation algorithm is shown in Algorithm 4.

Algorithm 4 Seed generation algorithm based on code slicing and large language models

Require: Original code C, optimized line set L_{opt}
Ensure: Generated initial seed set S_{gen}

Phase 1: Code Slicing
for each line $l_i \in L_{\text{opt}}$ **do**
  $B_i \leftarrow \text{CFBoundaryAnalysis}(C, l_i)$ ▷ Control flow boundary analysis
  $S_i \leftarrow \text{SliceConstruction}(C, B_i)$ ▷ Construct code slice
  $T_i \leftarrow \text{Annotate}(S_i, l_i)$ ▷ Annotate optimization line context
end for
Phase 2: Seed Generation
Initialize seed pool $S_{\text{pool}} \leftarrow \emptyset$
for each slice T_i **do**
  $P_{\text{prompt}} \leftarrow \text{FormatPrompt}(T_i)$ ▷ Format prompt
  $R_{\text{llm}} \leftarrow \text{QueryLLM}(P_{\text{prompt}})$ ▷ Query LLM API
  $S_{\text{cand}} \leftarrow \text{ParseResponses}(R_{\text{llm}})$ ▷ Parse candidate seeds
  $S_{\text{valid}} \leftarrow \text{ValidateSeeds}(S_{\text{cand}})$ ▷ Validate seed format
  $S_{\text{pool}} \leftarrow S_{\text{pool}} \cup S_{\text{valid}}$
end for
$S_{\text{gen}} \leftarrow S_{\text{pool}}$ ▷ Generate final seed set
return S_{gen}

3.6. Module Collaboration and Overall Process

RustOptFuzz modules work closely together through data flow and control flow to form a complete closed loop of vulnerability detection. The test case generation module

continuously provides the system with a high-quality, highly relevant test code; the potential vulnerability screening module performs static analysis on each test case, accurately marking and optimizing relevant areas and high-risk code lines; the targeted fuzz testing module targets these areas, automatically generates and screens high-quality seeds, performs targeted mutations and executions, and quickly focuses on potential vulnerability points. The entire process is highly automated, greatly improving the efficiency and accuracy of vulnerability detection.

3.7. Summary

The RustOptFuzz system integrates multiple technologies, including test case generation driven by large language models, vulnerability screening based on static analysis and rule matching, and directed fuzz testing focused on optimization-related regions. Through a modular and automated architectural design, RustOptFuzz enhances the efficiency and accuracy of detecting Rustc optimization vulnerabilities.

4. RustOptFuzz System Implementation

This chapter elaborates on the implementation process of the RustOptFuzz system, covering the specific methods used in core modules such as test case generation, potential vulnerability screening, and targeted fuzz testing. It also details key algorithmic aspects, the construction of the experimental environment, and the technical challenges encountered along with their solutions. Through an in-depth analysis of each module, this chapter demonstrates RustOptFuzz's technical innovations and engineering capabilities in the field of compiler optimization vulnerability detection.

4.1. Test Case Generation Method Based on LLM

Test case generation is the starting point of the RustOptFuzz system, aiming to produce a Rust code that is highly relevant to a specific optimization pass and can effectively trigger optimization behavior. To achieve this, the system employs an automated generation method based on an LLM, combined with prompt engineering and a multi-stage screening mechanism, which significantly improves the quality and relevance of test cases. The test case generation workflow is illustrated in Figure 2.

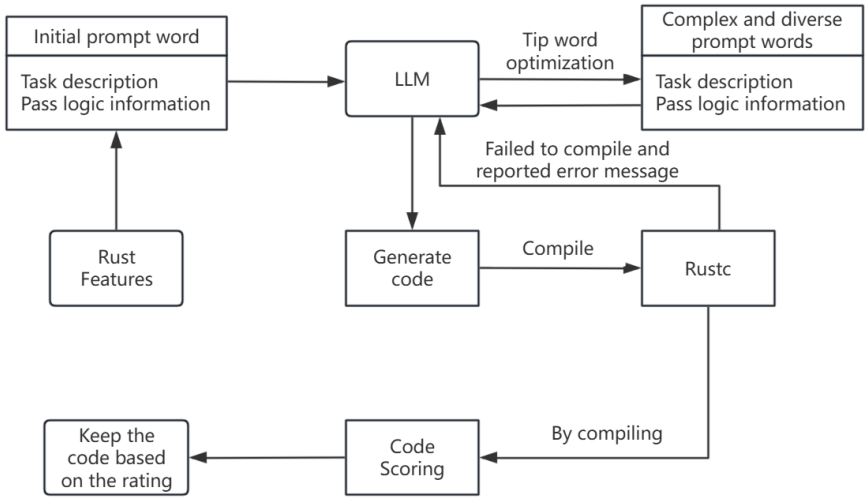

Figure 2. Test case generation workflow.

4.1.1. Prompt Word Template and Optimization Algorithm

First, the system designs a structured prompt template that includes the task description, optimization pass logic, Rust language features, and code generation constraints.

For example, the task description might be "generate a piece of Rust code to test the loop unrolling optimization pass," while the optimization pass section details the principles and applicable scenarios of loop unrolling.

The Rust features section can specify elements such as generics, traits, and unsafe code, and the constraints section limits aspects like code length and complexity. During implementation, Python scripts automatically assemble the template content, selecting elements from the pass library and feature library through random sampling to fill the template. Subsequently, an LLM (such as DeepSeek-Coder-v2) is used to generate initial prompt tokens. To enhance diversity and quality, the system adopts a temperature control strategy: low temperature (e.g., 0) generates basic prompt tokens, while high temperature (e.g., 1) generates diversified prompt tokens. All generated prompts are scored either manually or automatically to select the best candidates.

4.1.2. Code Generation and Multi-Stage Screening

Based on the optimized prompt words, LLM generates a Rust test code in batches. To ensure code quality, the system has designed the following screening process:

1. Compilation verification: Call the Rustc compiler to automatically compile and generate a code, capture compilation errors and feedback to the generation module, and automatically fix common syntax errors.
2. Optimization strength evaluation: By analyzing MIR and LLVM IR, the number of lines of a code that are actually affected by the optimization pass is counted, and the code that can effectively trigger the optimization is screened out.
3. Complexity analysis: Use the LLVM Analysis API to evaluate the complexity of control flow, data flow, and call relationship, and eliminate an overly simple or invalid code.

In the end, only a high-quality code that passes the entire screening process will be retained for subsequent testing.

4.1.3. Technical Difficulties and Solutions

Difficulty 1: The grammatical and semantic correctness of the generated code is difficult to guarantee.

Solution: Introduce automated compilation verification and error repair mechanisms, combined with multiple rounds of generation and screening, to significantly improve the code pass rate.

Difficulty 2: How to ensure the strong correlation between the generated code and the optimization pass.

Solution: Embed optimization pass logic in the prompt words, and filter out the code that can really trigger the target pass through optimization strength evaluation.

4.2. Potential Vulnerability Screening Method Based on Static Analysis and Rule Matching

The core task of this module is to accurately identify code regions related to optimization, reduce the scope of testing, and enhance the efficiency of vulnerability detection. The system integrates static analysis, control flow analysis, stubs, rule matching, and other techniques to conduct multi-level screening of MIR, LLVM IR, and unsafe code blocks. The static analysis workflow is illustrated in Figure 3.

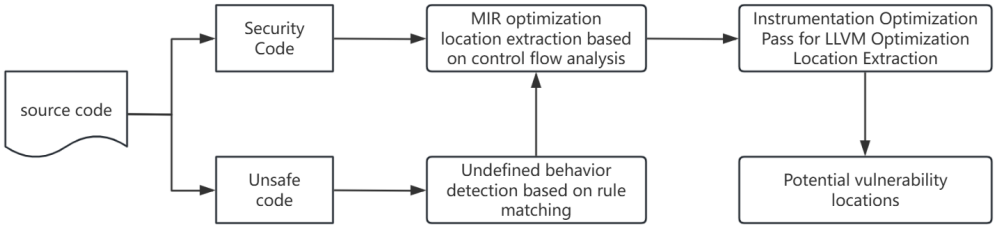

Figure 3. Static analysis workflow.

In the MIR layer optimization area screening stage, the system first parses the MIR before and after optimization to extract functions, basic blocks, statements, and their source code mappings. Through hash comparison, identify new, deleted, and modified basic blocks. Subsequently, traverse the affected basic blocks, extract statements containing source code location information, and finally output the source code line set affected by the optimization.

In the LLVM IR layer stub and analysis phase, the system inserts stubs at key optimization locations through custom passes to record the basic blocks involved in each optimization. Using LLVM's DebugInfo API, the basic blocks are mapped back to the Rust source code lines to achieve precise positioning of the optimization area. The instrumentation pass is implemented in C++ and supports compatibility with mainstream LLVM versions.

In the unsafe code rule matching phase, for Rust's unsafe code blocks, the system detects undefined behaviors such as null pointer dereference, array out-of-bounds, and type conversion based on AST traversal and rule matching. A series of rules are defined to match related undefined behaviors and mark the corresponding areas.

The mapping problem between an IR and Rust source code is solved by locating the corresponding code lines of functions and their parent functions based on the filtered basic blocks according to the debugging information.

4.3. Directed Fuzz Testing Method Based on Optimization Area Correlation

The directed fuzz testing module is one of the core innovations of RustOptFuzz. Its goal is to use optimization area information and high-quality initial seeds to efficiently discover misoptimization vulnerabilities. This module includes three parts: initial seed generation, seed queue management, and seed selection strategy. A directed fuzzy testing structure diagram based on optimized correlation is shown in Figure 4.

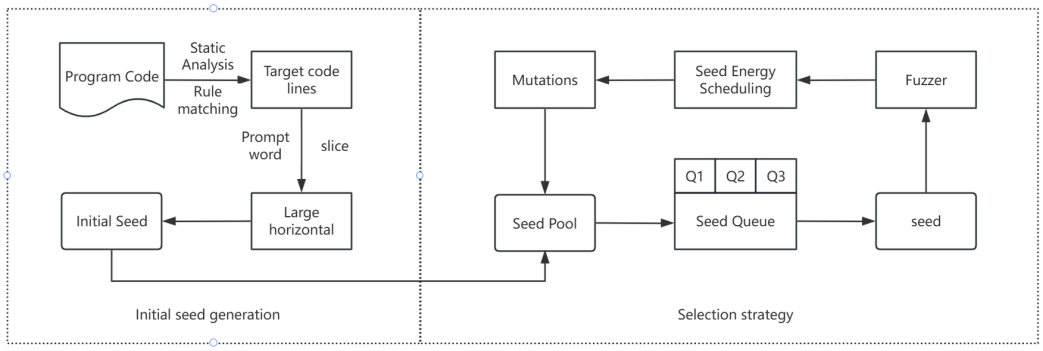

Figure 4. Directed fuzzy testing structure diagram based on optimized correlation.

In the context-aware initial seed generation phase, the system uses a context-aware code slicing algorithm to extract code snippets containing control flow boundaries to

optimize code lines and ensure the semantic integrity of the slices. Then, LLM is used to generate high-quality initial seeds based on the slice content. The generation process covers slice construction, prompt word formatting, LLM call, seed parsing, and verification, and finally forms a diverse and highly relevant seed pool.

In the multi-level seed queue management phase, in order to improve test efficiency, the system designs a three-level priority queue: Q1 (high priority) stores seeds closest to the target, Q2 (secondary priority) stores seeds with the highest optimized path coverage, and Q3 (low priority) stores other seeds. The queue content is dynamically adjusted in combination with the AFL-COV coverage tool to ensure that high-potential seeds are processed first.

In the dynamic seed selection and annealing strategy stage, seed selection uses annealing probability control to dynamically adjust the probability of skipping non-priority seeds. In the initial high-temperature stage, high-priority seeds are prioritized, and the high temperature gradually decreases in the later stage and turns to uniform selection. Specifically, the initial skip probability of executed seeds or non-priority seeds is high (such as 99%) and then gradually decreases. Unexecuted seeds are prioritized to explore new paths. This strategy effectively balances exploration and utilization and improves vulnerability discovery efficiency.

In response to the technical difficulty of how to balance new path exploration and the use of high-quality seed nodes, the system uses the annealing algorithm to dynamically adjust the selection probability and optimizes resource allocation by combining coverage and distance dual indicators.

4.4. Experimental Environment Construction

To ensure the reproducibility of the experiment and the efficient operation of the system, RustOptFuzz is developed and tested in the following environment:

Hardware environment: AMD Ryzen 9 5900HX 3.3 GHz, 32 GB memory, 1 TB SSD;

Operating system: Ubuntu 22.04 64bit;

Compiler and tool chain: Rustc 1.53.0, LLVM 12.0.1, Python 3.8, Docker for environment isolation;

LLM API: DeepSeek-Coder-v2, API calls are automatically managed by Python scripts

4.5. Key Code Snippet Examples

The following are pseudocode examples of some key implementations:

Test case generation and screening:

Prompt-based Code Generation (Algorithm 5):

Algorithm 5 Prompt-Based numerical order. Code Generation

```
for each prompt ∈ prompt_list do
    code ← llm.generate_code(prompt)
    if compile(code) then
        if evaluate_optimization(code) > threshold then
            save(code)
        end if
    end if
end for
```

MIR Optimization Area Screening (Algorithm 6):

Algorithm 6 MIR Optimization Area Screening

```
for each func ∈ mir_file.functions do
    for each bb ∈ func.basic_blocks do
        if bb ∈ diff_blocks then
            for each stmt ∈ bb.statements do
                if stmt.has_src_loc() then
                    affected_lines.add(stmt.src_loc)
                end if
            end for
        end if
    end for
end for
```

Seed Queue Management and Selection (Algorithm 7):

Algorithm 7 Seed Queue Management and Selection

```
for each queue ∈ [Q1, Q2, Q3] do
    for each seed ∈ queue do
        if pending_favored > 0 then
            if seed.was_fuzzed ∨ ¬seed.is_favored then
                if random() < P_skip then
                    continue
                end if
            end if
        else
            if ¬seed.is_favored then
                if seed.was_fuzzed ∧ random() < 0.95 then
                    continue
                else if random() < 0.75 then
                    continue
                end if
            end if
        end if
        fuzz(seed)
    end for
end for
```

4.6. Summary of Technical Difficulties and Innovations

During the implementation process, RustOptFuzz employed several key technologies, including ensuring the semantic correctness of a code generated by large language models, precise screening of optimization areas, and dynamic management of seed priorities. By introducing innovative mechanisms such as multi-stage screening, the integration of static and dynamic analysis, and annealing probability scheduling, the system has significantly improved test case quality, vulnerability detection efficiency, and resource utilization. The standardization of the experimental environment and the use of automated scripts further guarantee the reproducibility and practical applicability of the system.

In summary, RustOptFuzz has established an efficient system for detecting vulnerabilities in Rustc optimizations through a modular and automated technical approach, providing a solid foundation for future research and engineering applications in this domain.

5. Evaluation

This chapter comprehensively evaluates the performance of the RustOptFuzz system, covering experimental design, experimental results analysis, comparison with mainstream fuzz testing tools, and in-depth discussion of algorithm parameters and test case generation effectiveness. Through systematic experiments, RustOptFuzz's advantages in vulnerability discovery, detection efficiency, and reliability are demonstrated.

5.1. Discovered Vulnerabilities

In order to evaluate the ability of RustOptFuzz in actual vulnerability mining, we conducted systematic experiments on our own dataset and five real Rust programs. Our own dataset consists of 200 test files generated by a large language model, and the real programs are selected from open-source projects on GitHub that are known to have optimization vulnerabilities. All experiments are conducted in a unified hardware environment to ensure the comparability of the results.

First, to verify that our own dataset has statistical significance, this paper conducts statistical analyses on our own dataset from four dimensions: Number of Functions, Lines of Code, Number of Variable Types, and Number of Loops. The final experimental results are shown in Figure 5. It can be seen that, starting with the Number of Functions, the distribution of this feature in the generated test files presents a certain degree of concentration and dispersion. For Number of Functions, one-way ANOVA ($p < 0.05$) reveals significant variability among samples, indicating inherent diversity in functional complexity. For Lines of Code, ANOVA ($p < 0.05$) confirms significant differences in code scale, consistent with real-world programming diversity. Regarding Number of Variable Types, ANOVA ($p < 0.05$) shows notable variation in variable type usage, reflecting meaningful data handling complexity. For Number of Loops, ANOVA ($p < 0.05$) indicates significant differences in loop frequency, aligning with practical control flow variations. Overall, significant ANOVA results across all dimensions confirm the dataset's statistically meaningful variability, validating its representativeness for subsequent experiments.

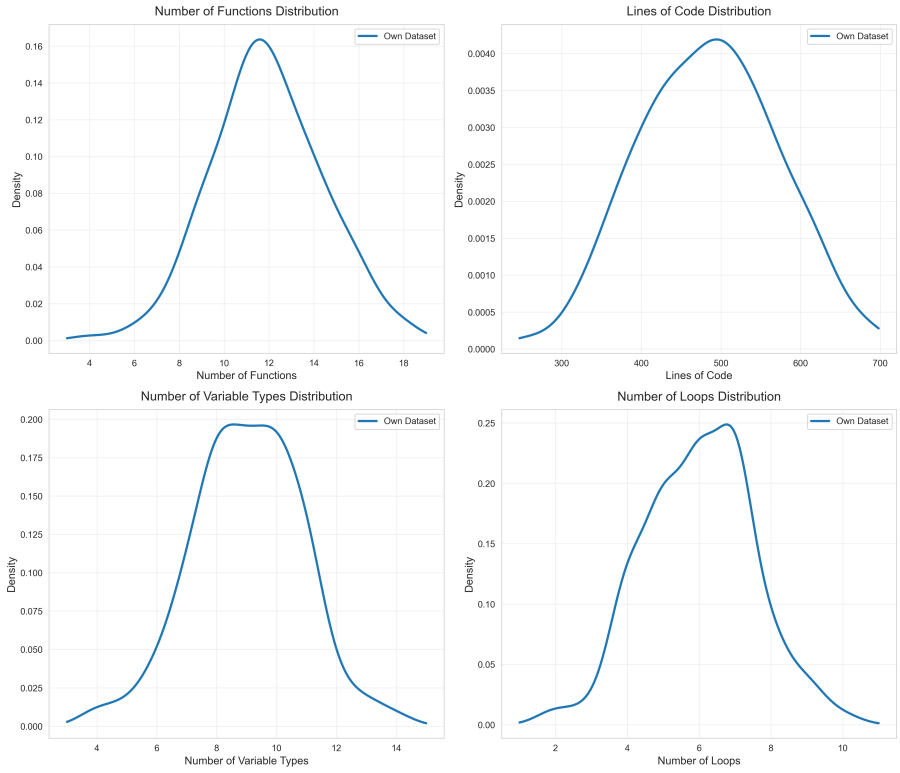

Figure 5. Statistical analysis of own dataset.

As shown in Figure 6 and Table 1, the experimental results show that the average number of vulnerabilities discovered by RustOptFuzz on our own dataset is 2.8, which is significantly higher than 2.4 of WindRanger and 1.8 of AFL.rs. In terms of standard deviation, RustOptFuzz is 0.447, showing high stability. Further analysis shows that RustOptFuzz can find more deep vulnerabilities related to the optimization pass, especially in scenarios involving complex control flows and unsafe code blocks.

Figure 6. Distribution comparison of vulnerabilities discovered by different tools (scatter plot).

Table 1. Average number of vulnerabilities discovered on our own datasets.

Tool	Average Number of Vulnerabilities	Standard Deviation
RustOptFuzz	2.8	0.447
WindRanger	2.4	0.547
AFL.rs	1.8	0.447

To verify the stability of the method proposed in this paper, the above 200 test files were randomly divided into 10 groups, and the vulnerability detection capabilities of three tools (RustOptFuzz, WindRanger, and AFL.rs) were evaluated, respectively. The final experimental results are shown in Figure 7. It can be observed that RustOptFuzz consistently outperforms the other two tools across all 10 test groups, with its average number of detected vulnerabilities per group ranging from 2.55 to 3.05, maintaining a clear lead. WindRanger demonstrates a moderate performance, with its per-group averages fluctuating between 2.15 and 2.6, which is stably lower than RustOptFuzz but significantly higher than AFL.rs. AFL. rs shows the lowest vulnerability detection capability among the three, with per-group averages varying from 1.6 to 2.0, forming a distinct gradient with the other two tools.

In real program testing, RustOptFuzz detected a total of 5 known optimization vulnerabilities, and the vulnerability reproduction time was significantly shorter than that of the comparison tools. For example, in the Simplifycfg.rs program, RustOptFuzz can accurately locate and reproduce the control flow misoptimization vulnerability introduced by the LLVM SimplifyCFG pass, and the relevant call stack is highly consistent with the optimization area screened by static analysis. This result shows that RustOptFuzz not only

has strong vulnerability discovery capabilities, but also can effectively assist in vulnerability location and reproduction.

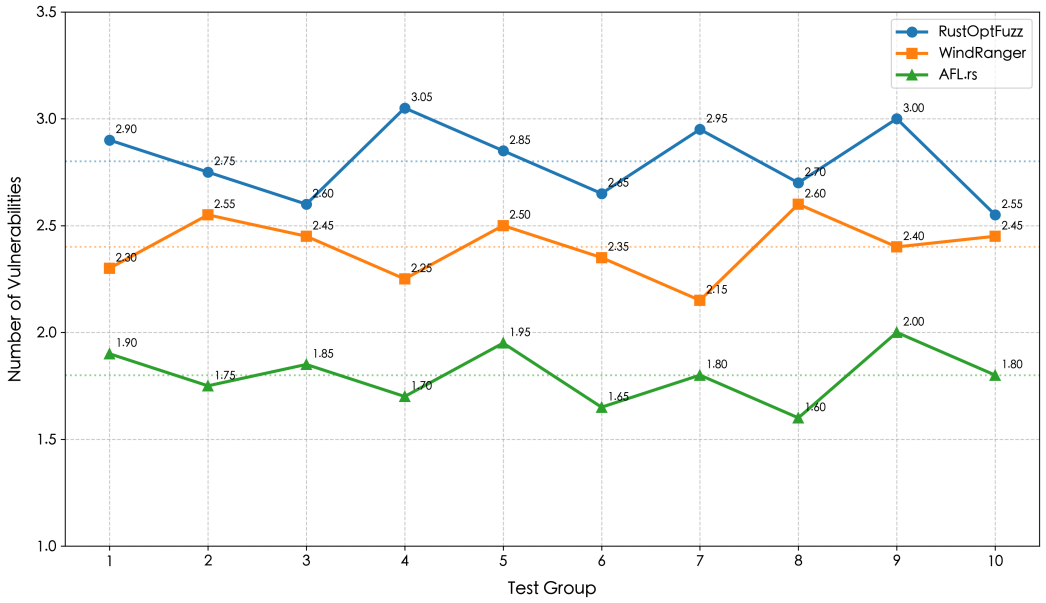

Figure 7. Comparison of vulnerability discovery capabilities of different tools in test groups.

5.2. Comparison with Existing Fuzz Testing Methods

To comprehensively evaluate the performance of RustOptFuzz, we selected WindRanger and AFL.rs as comparison objects, representing the mainstream implementations of directed fuzz testing and traditional coverage-oriented fuzz testing, respectively. All tools were run on the same dataset and experimental environment, and each group of experiments was repeated five times, and the average value was taken as the final result.

In terms of vulnerability discovery capabilities, RustOptFuzz outperforms the comparison tools on both its own datasets and real programs. Taking its own dataset as an example, RustOptFuzz finds 2.8 vulnerabilities every 30 min on average, WindRanger finds 2.4, and AFL.rs finds 1.8. As shown in Table 2, on real programs, both RustOptFuzz and WindRanger can find 5 vulnerabilities, but the average detection time of RustOptFuzz is 112 min, which is lower than WindRanger's 123 min and AFL.rs's 150 min.

Table 2. Number of vulnerabilities discovered on five real programs.

Tool	Number of Vulnerabilities	Elapsed Time (min)
RustOptFuzz	5	112
WindRanger	5	123
AFL.rs	4	150

As shown in Table 3, in terms of resource consumption and efficiency, RustOptFuzz's CPU utilization rate reaches 88%, the peak memory is 2.6 GB, and the seed queue size is 852, showing good resource management capabilities. Although AFL.rs has the largest seed queue, it also has the highest memory consumption, and its efficiency in targeted vulnerability detection is not as good as RustOptFuzz and WindRanger.

Table 3. Resource consumption and efficiency.

Tool	CPU Utilization	Peak Memory (GB)	Seed Queue Size
RustOptFuzz	88%	2.6	852
WindRanger	83%	2.2	783
AFL.rs	85%	3.2	1626

In the line coverage evaluation, RustOptFuzz has a higher coverage rate than WindRanger on most programs, indicating that its optimized seed queue and initial seed generation mechanism can explore the target code area more effectively. The hierarchical queue management and the diversified seeds generated by the large model jointly improve the coverage of test cases.

In terms of vulnerability reproduction ability, RustOptFuzz and WindRanger can reproduce all known vulnerabilities within 30 min, but RustOptFuzz reproduces faster and performs more stably in complex optimization scenarios. Due to the lack of a directional mechanism, some vulnerabilities in AFL.rs cannot be reproduced within the limited time.

5.3. Impact of Algorithm Parameters

The ablation experiment further verifies the effectiveness of each key technology. After removing the initial seed of the large model, the number of vulnerability detections dropped from 2.8 to 2.1, the detection time increased by 14.3%, and the line coverage decreased by 11.2 percentage points. When the traditional two-level queue was used instead of the three-level queue, the number of vulnerability detections dropped to 2.4, the detection time increased by 9.8%, and the coverage decreased by 5.1 percentage points. The basic seed selection strategy also leads to performance degradation. These results show that the initial seed of the large model, the queue management of the optimized region association, and the improved seed selection strategy all have a significant effect on improving the system performance.

5.4. Effectiveness of Test Case Generation

The effectiveness of test case generation directly determines the coverage capability and vulnerability discovery efficiency of fuzz testing. To this end, we systematically evaluated the test cases generated by RustOptFuzz, including indicators such as compilation pass rate, optimization pass trigger rate, and code diversity.

As shown in Table 4, among the 1000 test cases generated by the large model, the compilation pass rate was 56.1%, the optimization pass trigger rate was as high as 90.32%, and the proportion of samples with optimized lines reaching or exceeding the threshold was 95.21%. These results show that RustOptFuzz can generate a large number of high-quality test cases that are highly correlated with the optimization pass, providing a solid foundation for subsequent vulnerability detection.

Table 4. Compile and optimize recognition results.

Index	Calculation Formula	Result
Compilation pass rate	Successful compilations/total builds $*$ 100%	56.1%
Optimization evaluation	Optimize the proportion of samples with the number of rows $\geq$ the threshold	95.21%
Trigger optimization of pass rate	Number of successful triggers/number of used passes	90.32%

Further diversity analysis shows that the prompt words with an edit distance greater than 100 account for 73%, and the prompt words with a cosine similarity of less than 0.5 account for 51.1%, indicating that the generated prompt words and codes have high diversity at both the character and semantic levels. The cosine similarity and Jaccard similarity of the codes between groups are both concentrated in the lower range, indicating that the generated code covers a rich range of optimization scenarios and boundary conditions.

Among them, the cosine similarity is calculated by the following Formula (2):

$$cosine(A, B) = \frac{A \cdot B}{\|A\|\|B\|} \tag{2}$$

The Jaccard similarity is calculated by the following Formula (3):

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \tag{3}$$

In actual vulnerability detection, the test cases generated by RustOptFuzz can effectively trigger multiple optimization passes, with an average coverage of 80%, significantly higher than AFL.rs and WindRanger. This advantage is particularly evident in complex optimization scenarios and tests involving advanced features of Rust.

5.5. Summary

In summary, RustOptFuzz is superior to existing mainstream fuzz testing tools in terms of vulnerability discovery capabilities, detection efficiency, resource management, and test case generation. The system achieves efficient detection of Rustc optimization vulnerabilities through innovative technologies such as test case generation driven by a large language model, multi-level static analysis, and optimization-related directional fuzz testing. The experimental results fully verify the effectiveness, efficiency, and reliability of RustOptFuzz, providing strong support for subsequent compiler security research and engineering practice.

6. Related Work

In the modern software development process, testing is the core means to maintain software quality. As a key basic software, the test input of the compiler is usually a source code written in the target language. The core goal of compiler optimization vulnerability detection is to discover unexpected behaviors generated after the program is compiled when the user enables a specific combination of optimization options. However, the current test is basically based on a semantically generated random code, which makes it difficult to cover the optimization pass in a targeted manner, resulting in severe challenges in optimization vulnerability detection and verification. Existing compiler test program generation methods are mainly divided into three categories: specification-based methods (such as Csmith [9], RustSmith [10], and YARPGen [11]) randomly generate a code through grammatical semantic rules; machine learning-based methods (such as DeepSmith [12], DeepFuzz [13], and DSmith [14]) use deep learning technology to analyze code features to generate programs; and mutation-based methods (such as Orion [15], Athena [16], and Hermes [17]) generate equivalent variants by modifying existing test programs. While these methods improve test coverage, they still have problems such as low correlation between a generated code and optimization pass and insufficient semantic perception.

Rustc performs optimizations at the MIR and LLVM IR layers during compilation, which may introduce optimization vulnerabilities. There are many studies on LLVM IR optimization verification. For example, Alive2 [18] uses an SMT solver to verify the semantic consistency of optimization pass and has found 47 new vulnerabilities in LLVM,

but there are performance bottlenecks in complex loop optimization scenarios. Crellvm [19] converts LLVM IR into Coq formal representation through a trusted compilation framework, generates optimization correctness proof, and ensures that the semantics of an optimized code is consistent with the source code. Directed fuzz testing focuses on specific areas in the program where vulnerabilities may exist, and improves the efficiency of vulnerability discovery through target-oriented seed selection and energy scheduling. For example, AFLGO [20] calculates the distance from the basic block to the target based on the control flow graph, and prioritizes seeds closer to the target. Hawkeye [21] optimizes distance measurement and seed priority, Lolly [22] adopts sequence coverage-oriented strategy, and WindRanger [23] improves directional capability through deviation block distance and data-flow-sensitive mutation. However, existing methods still have defects such as insufficient test code targeting, high optimization verification complexity, and low fuzz testing efficiency, making it difficult to efficiently detect Rustc optimization vulnerabilities.

Among the existing methods, the test program generation methods based on specifications, machine learning, and mutation have problems such as low correlation with optimization pass and insufficient semantic perception. Optimization verification tools such as Alive2 and Crellvm have performance bottlenecks when dealing with complex scenarios. Directed fuzz testing methods also face challenges such as low initial seed quality and insufficient consideration of optimization area correlation. The innovation of this study is to propose a test case generation method based on a large language model, combine static analysis and rule matching for potential vulnerability screening and a directed fuzz testing strategy based on optimization area correlation, design and implement the RustOptFuzz system, and effectively improve the correlation between test cases and optimization pass, vulnerability location efficiency, and fuzz testing targeting. Experiments show that this method has significantly improved vulnerability discovery capabilities and reproduction efficiency compared with existing tools.

7. Conclusions and Future Work

Rust has rapidly developed in the field of system programming due to its advantages in memory safety, high performance, and concurrency, especially in industrial control scenarios. However, the complexity of its compiler optimizations makes it prone to vulnerabilities, which may cause the real-time performance of the industrial control system to decrease or safety risks. Existing testing methods suffer from shortcomings such as strong randomness in test cases, lack of optimization area screening mechanisms, and insufficient consideration of optimization correlations in directional fuzz testing. This study proposes a directional fuzz testing method based on large language model-driven test case generation, static analysis and rule-based vulnerability screening, and optimization area correlation guidance, and designs and implements the RustOptFuzz system. Experimental results demonstrate that its vulnerability detection capability on both proprietary datasets and real-world programs is improved by 16% to 50%, effectively validating the feasibility and superiority of the method.

Although this study has achieved notable results, some limitations remain, including the limited ability of large models to generate a complex Rust code, a high false-positive rate in static analysis, and reliance on single-dimensional seed queue evaluation metrics. Future work will focus on improving the quality of a code generated by large models through phased generation strategies, introducing dynamic analysis and multi-dimensional evaluation metrics to optimize vulnerability screening and seed selection, and further enhancing optimization area correlation analysis to improve the efficiency and accuracy of Rustc optimization vulnerability detection, providing a more solid guarantee for the safe and stable operation of industrial control systems.

Author Contributions: Methodology, K.X., J.W. and Y.W.; Software, K.X. and J.W.; Validation, L.C.; Investigation, L.C.; Writing—review & editing, Y.W. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the National Key R&D Program of China, grant number 2022YFB3104300.

Data Availability Statement: The data presented in this study are available on request from the corresponding author.

Conflicts of Interest: The authors declare no conflict of interest.

Abbreviations

The following abbreviations are used in this manuscript:

AST	abstract syntax tree
CFG	control flow graph
CoT	chain-of-thought
DBB	deviation basic block
DGF	directed greybox fuzzing
HIR	high-level intermediate representation
iCFG	interprocedural control flow graph
LLM	large language model
LLVM IR	low-level virtual machine intermediate representation
MCMC	Markov chain Monte Carlo
MIR	mid-level intermediate representation
SMT	satisfiability modulo theories
SVF	static value-flow analysis

References

1. Aho, A.V.; Lam, M.S.; Sethi, R.; Lam, M. *Compilers: Principles, Techniques, and Tools*, 2nd ed.; Addison-Wesley Longman Publishing Co., Inc.: Boston, MA, USA, 2006.
2. Wang, X.; Zeldovich, N.; Kaashoek, M.F.; Solar-Lezama, A. A differential approach to undefined behavior detection. *ACM Trans. Comput. Syst.* **2015**, *33*, 1–29. [CrossRef]
3. Rust Blog. rustc-dev-guide. 2020. Available online: https://blog.rust-lang.net.cn/inside-rust/2020/03/26/rustc-dev-guide-overview.html (accessed on 26 March 2020).
4. GNU Project. Gcc, the GNU Compiler Collection. 2024. Available online: https://gcc.gnu.org/ (accessed on 11 July 2025).
5. Lattner, C.; Adve, V. LLVM: A compilation framework for lifelong program analysis & transformation. In *CGO '04: Proceedings of the International Symposium on Code Generation and Optimization: Feedback-Directed and Runtime Optimization*; IEEE Computer Society : San Jose, CA, USA, 2004; Volume 75.
6. Sun, C.; Le V.; Zhang, Q.; Su, Z. Toward understanding compiler bugs in GCC and LLVM. In *ISSTA 2016: Proceedings of the 25th International Symposium on Software Testing and Analysis*; Association for Computing Machinery: New York, NY, USA, 2016; pp. 294–305. [CrossRef]
7. Rust—The Safe, Productive, Fast Language. Rust-Lang/Rust: List of Open Issues Labeled 'a-mir-opt'. 2024. Available online: https://github.com/rust-lang/rust/issues?q=is%3Aissue%20state%3Aopen%20label%3AA-mir-opt (accessed on 26 March 2020).
8. Rust—The Safe, Productive, Fast Language. Rust-Lang/Rust: List of Open Issues Labeled 'a-llvm'. 2024. Available online: https://github.com/rust-lang/rust/issues?q=is%3Aissue%20state%3Aopen%20label%3AA-LLVM (accessed on 26 March 2020).
9. Yang, X.; Chen, Y.; Eide, E.; Regehr, J. Finding and understanding bugs in C compilers. In *PLDI '11: Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation*; Association for Computing Machinery: New York, NY, USA, 2011; pp. 283–294. [CrossRef]
10. Sharma, M.; Yu, P.; Donaldson, A.F. Rustsmith: Random differential compiler testing for Rust. In *ISSTA 2023: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, Seattle, WA, USA, 17–21 July 2023* ; Association for Computing Machinery: New York, NY, USA, 2023; pp. 1483–1486. [CrossRef]
11. Livinskii, V.; Babokin, D.; Regehr, J. Random testing for C and C++ compilers with YarpGen. *Proc. ACM Program. Lang.* **2020**, *4*, 1–25. [CrossRef]

12. Cummins, C.; Petoumenos, P.; Murray, A.; Leather, H. Compiler fuzzing through deep learning. In *ISSTA 2018: Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis, Amsterdam, The Netherlands, 16–21 July 2018*; Association for Computing Machinery: New York, NY, USA, 2018; pp. 95–105. [CrossRef]
13. Liu, X.; Li, X.; Prajapati, R.; Wu, D. Deepfuzz: Automatic generation of syntax valid C programs for fuzz testing. In *AAAI'19/IAAI'19/EAAI'19: Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence and Thirty-First Innovative Applications of Artificial Intelligence Conference and Ninth AAAI Symposium on Educational Advances in Artificial Intelligence*; AAAI Press: Honolulu, HI, USA, 2019. [CrossRef]
14. Xu, H.; Wang, Y.; Fan, S.; Xie, P.; Liu, A. Dsmith: Compiler fuzzing through generative deep learning model with attention. In Proceedings of the 2020 International Joint Conference on Neural Networks (IJCNN), Glasgow, UK, 19–24 July 2020; pp. 1–9. [CrossRef]
15. Le V.; Afshari, M.; Su, Z. Compiler validation via equivalence modulo inputs. In *PLDI '14: Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation*; Association for Computing Machinery: New York, NY, USA, 2014; pp. 216–226. [CrossRef]
16. Le V.; Sun, C.; Su, Z. Finding deep compiler bugs via guided stochastic program mutation. *Sigplan Not.* **2015**, *50*, 386–399. [CrossRef]
17. Sun, C.; Le V.; Su, Z. Finding compiler bugs via live code mutation. *Sigplan Not.* **2016**, *51*, 849–863. [CrossRef]
18. Lopes, N.P.; Lee, J.; Hur, C.K.; Liu, Z.; Regehr, J. Alive2: Bounded translation validation for LLVM. In *PLDI 2021: Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*; Association for Computing Machinery: New York, NY, USA, 2021; pp. 65–79. [CrossRef]
19. Kang, J.; Kim, Y.; Song, Y.; Lee, J.; Park, S.; Shin, M.D.; Kim, Y.; Cho, S.; Choi, J.; Hur, C.K.; et al. Crellvm: Verified credible compilation for LLVM. *SIGPLAN Not.* **2018**, *53*, 631–645. [CrossRef]
20. Böhme, M.; Pham, V.T.; Nguyen, M.D.; Roychoudhury, A. Directed greybox fuzzing. In *CCS '17: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*; Association for Computing Machinery: New York, NY, USA, 2017; pp. 2329–2344. [CrossRef]
21. Chen, H.; Xue, Y.; Li, Y.; Chen, B.; Xie, X.; Wu, X.; Liu, Y. *Hawkeye: Towards a desired directed grey-box fuzzer. In CCS '18: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*; Association for Computing Machinery: New York, NY, USA, 2018; pp. 2095–2108. [CrossRef]
22. Liang, H.; Zhang, Y.; Yu, Y.; Xie, Z.; Jiang, L. Sequence coverage directed greybox fuzzing. In *ICPC '19: Proceedings of the 27th International Conference on Program Comprehension*; IEEE Press: Montreal, QC, Canada, 2019; pp. 249–259. [CrossRef]
23. Du, Z.; Li, Y.; Liu, Y.; Mao, B. Windranger: A directed greybox fuzzer driven by deviation basic blocks. In Proceedings of the 2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE), Pittsburgh, PA, USA, 25–27 May 2022; pp. 2440–2451. [CrossRef]